Loop Programming with `/loop`

This document describes the **`/loop` goal-driven loop programming extension** (spec: `specs/agentloop`). `/loop` lets you *program* the agent's behaviour with a declarative specification: a goal (success state), an optional verification command, a tool surface, scope boundaries, and budget limits. The agent then keeps its Reason -> Act -> Observe loop running until the specification says it is done.

`/loop` is **not** the core per-turn agent loop. The core loop is always running for every turn (see `docs/howtos/reactagent.md`). `/loop` *wraps* that loop: it injects stop conditions, a verification gate, workspace capture/rollback, and a "Goal Loop" section into the system prompt at fixed safe points.

Contents

1. What `/loop` adds
2. Architecture overview
3. Starting a loop
4. The interactive setup dialog
5. Writing a goal
6. The LoopSpec fields
7. Stop conditions and termination statuses
8. The verification gate
9. Restrictions: tool set, read-only, scope
10. Checkpoints and rollback
11. Configuration options
12. Commands and keyboard surface
13. Worked examples
14. Events and telemetry
15. Related documentation

---

1. What `/loop` adds

Without `/loop`, a turn works like this: you send a message, the model streams a response, any requested tools execute, results are appended, and the loop ends when the model answers without tool calls. You then prompt again for the next step.

With `/loop`, you declare an end state up front and the agent iterates autonomously toward it:

- **Goal** – the success state, expressed in plain text. Required.
- **Verification command** (optional) – a shell command whose *exit status* decides whether the goal is actually achieved. This is the strongest stop condition: a text-only "I'm done" response does not count until the command exits 0.
- **Tool set** (optional) – restricts which tools the loop may use.
- **Read-only constraints** (optional) – glob patterns for paths the loop must not write.
- **Scope boundaries** (optional) – glob patterns limiting where the loop may act at all.
- **Budgets** – a maximum iteration count and/or a maximum accumulated token cost.

Everything is enforced inside the normal agent loop at safe points, so a running loop remains interruptible and observable like any other turn.

2. Architecture overview

Layer	Location	Contents
State + enforcement	`crates/ragent-agent/src/session/loop_spec.rs`	`LoopSpec`, `LoopTracker`, `StopCondition`, `LOOP_WRITE_TOOLS`, `LOOP_ALWAYS_ALLOWED_TOOLS`, the `deny_reason` predicate
Runtime wiring	`crates/ragent-agent/src/session/processor.rs`	`SessionProcessor::start_loop`, `terminate_loop_inner`, `request_loop_interrupt`, `ensure_pre_loop_capture`, `rollback_loop`, `clear_loop`, `clear_loop_captures`
Verification	`crates/ragent-agent/src/session/verification.rs`	`run_verification_command`, `VerificationOutcome`, `verification_failure_observation`
Capture / rollback	`crates/ragent-agent/src/session/git_state.rs`	Pre-loop snapshot, git state, change summary, `compute_change_summary`
Prompt injection	`crates/ragent-agent/src/session/prompt.rs`	`build_loop_section` renders the `## Goal Loop` block into the turn system prompt
One-shot parsing	`crates/ragent-tui/src/app/slash_handlers.rs`	`handle_loop_command` (`/loop <agent> <goal...>`)
Interactive dialog	`crates/ragent-tui/src/app/loop_setup.rs` (+ `loop_dialog/` module dir)	`LoopSetupField`, `LoopSetupState`, `build_spec_from_state`, key handling
Termination rendering	`crates/ragent-tui/src/app/helpers.rs`, `event_handler.rs`	`loop_termination_banner`, `LoopTerminated` / `LoopChangeSummary` handling, rollback offer
Config	`crates/ragent-config/src/config.rs`	`LoopConfig` serde section (`loop` key) and its defaults

`SessionProcessor::start_loop` accepts an already-built `LoopSpec`; argument parsing and validation live in the TUI layer.

3. Starting a loop

There are three ways to start a goal loop.

3.1 One-shot (slash command)

```
/loop <agent> <goal text...>
```

Everything up to the first whitespace is the agent preset; the remainder is the goal. Examples:

```
/loop coder make all unit tests pass
/loop general summarise every README under docs/ into one index
```

One-shot defaults (documented in the handler doc comment):

- Step limit: the `loop.max_steps` value from `ragent.json` (default **512**).
- Cost limit: the `loop.cost_limit` value from `ragent.json` (default: none).
- Checkpoints: **on**.
- No verification command, no scope, no read-only constraints, no tool-set restriction.

Three one-shot flags override the run's budgets (any position, `--flag value` or `--flag=value`; `--cost_limit` and `--cost-limit` are both accepted):

```
/loop coder --max-steps 10 make all unit tests pass
/loop coder "refactor the auth module" --max-steps=40 --cost_limit 200000
/loop general --timeout 30 ship the release notes
```

Flag	Effect
`--max-steps N`	Override the step budget for this run
`--cost_limit N` / `--cost-limit N`	Override the token-cost budget for this run
`--timeout N`	Override the checkpoint-prompt timeout (seconds) for this run

An invalid flag value (missing, non-numeric, or out of range) stops the loop from starting and reports the offending flag. All other richer knobs (verify, scope, read-only, tool set) are set in the interactive dialog.

Starting a loop with an empty goal shows an error naming the missing field and re-opens the setup dialog with the typed agent pre-selected:

```
From: /loop

**Error:** missing field: goal - a loop cannot start without a goal.

Usage: `/loop <agent> <goal text...>` or `/loop` for the setup dialog.
Re-opening the setup dialog with your agent ({agent}) pre-selected.
```

The status bar shows `loop: missing field: goal`.

3.2 Interactive dialog

```
/loop
```

Opens the `/loop setup` overlay where all nine fields can be set (Section 4). Pressing `Enter` validates and starts the loop; `Esc` cancels but keeps every entered value as a draft for the next open.

3.3 Preconditions

`App::start_goal_loop` rejects the start (status-bar messages, no loop started) when:

- No provider is configured: `loop: no provider configured – use /provider first`
- No model is selected: `loop: no model selected – use /model first`
- No active session: `[warn] No active session`

On success the message window shows `[loop {agent}] {goal}`, the working status becomes `loop running`, and one spawned task runs `processor.start_loop(&sid, spec)` followed by `processor.process_message(&sid, &goal_text, &agent, flag)`.

The **driver resolution** rule: the spec's agent preset wins; an unknown preset name falls back to the currently selected agent, so a typo never silently changes the driver.

---

4. The interactive setup dialog

4.1 Fields (in Tab order)

#	Field	Meaning	Notes
1	`agent`	Agent preset that drives the loop	Arrow keys select; selected agent is bold cyan
2	`goal`	Success state the loop must reach	**Required** (FR-005)
3	`verify cmd`	Verification command gating achievement	Optional (FR-007)
4	`scope`	Scope boundaries, comma-separated globs	Optional (FR-022)
5	`read-only`	Read-only constraints, comma-separated globs	Optional (FR-021)
6	`tools`	Restricted tool surface, comma-separated names	Optional (FR-008)
7	`max steps`	Step budget	Blank applies the config default (FR-013)
8	`cost limit`	Token cost budget	Blank applies the config default (FR-014)
9	`checkpoints`	Destructive-action checkpoint toggle	Toggled with `Space` or `Up`/`Down`; defaults **on**

`max steps` is prefilled from the on-disk `loop.max_steps` config value; `cost limit` is prefilled only when a value is configured.

4.2 Key bindings

Key	Action
`Tab` / `BackTab`	Next / previous field
`Up` / `Down`	`agent`: pick previous/next agent; `checkpoints`: toggle; text fields: move field
`Space`	Toggle `checkpoints` when focused
`Enter`	Validate and start the loop (draft slot cleared on success)
`Esc`	Close without starting; **all entered values are kept** as a draft (`loop: setup cancelled (values kept)`)
`Ctrl+V` / `Ctrl+W` / `Ctrl+K` / `Ctrl+U`	Paste / delete previous word / delete to end / clear line (in text fields)
Shift+Arrow	Select text (in text fields)

If validation fails (missing goal), the error is rendered inside the dialog in red: `error: missing field: goal - a loop cannot start without a goal`.

The footer hint line reads: `Tab next field - Up/Down navigate - Enter start - Esc cancel`.

---

5. Writing a goal

A goal is a *success state*, not a task description. Phrase it so that a software process (or a human reviewer) can tell unambiguously whether the state has been reached. The strongest form pairs the goal with a verification command:

Element	Weak	Strong
Goal	"improve the tests"	"all unit tests in `crates/ragent-tui` pass"
Verify cmd	–	`cargo test -p ragent-tui`
Goal	"clean up the docs"	"every `.md` file in `docs/` has a one-line summary in `docs/INDEX.md`"
Verify cmd	–	`python check_index.py`

Guidelines:

1. **State the end condition, not the journey.** The loop decides its own steps.
2. **Prefer a machine-checkable verify command.** Its exit status is what ends the loop; without one, the first tool-free answer ends it.
3. **Bound the blast radius.** Use `scope` to keep work inside a directory subtree and `read-only` to forbid writes to specific paths.
4. **Give a budget.** Unbounded loops burn tokens; set `max steps` (dialog or config) so a stuck run ends with a diagnosable status.

---

6. The LoopSpec fields

`LoopSpec` (`loop_state.rs`) is the serialisable specification. `agent` and `goal` are required. `verify_cmd`, `scope`, `read_only`, `tool_set`, and `checkpoint_timeout_secs` are omitted from JSON when unset; `max_steps` and `cost_limit` serialise as `null` when unset (both may also be omitted on input); `checkpoints` is always present (bool) and must be supplied when deserialising a spec.

Field	Type	Default	Purpose
`agent`	`String`	`"general"`	Agent preset driving the loop
`goal`	`String`	(empty)	The success state; required to start
`verify_cmd`	`Option<String>`	`None`	Verification command gating achievement; `None` means a no-tool-call response completes the loop directly
`scope`	`Vec<String>`	empty	Scope-boundary globs; empty means the working directory with no extra restriction
`read_only`	`Vec<String>`	(empty)	Read-only globs; writes to matching paths are denied and become observations
`tool_set`	`Vec<String>`	(empty)	Restricted tool surface; empty means all tools
`max_steps`	`Option<u32>`	`None`	Step budget; `None` resolves to the agent preset's `max_steps`, else the config default (512)
`cost_limit`	`Option<u64>`	`None`	Token-cost gate (accumulated input + output); `None` resolves to the config value, which defaults to no gate
`checkpoints`	`bool`	`true`	Force destructive-action checkpoints even when an allow rule would auto-approve

Field	Type	Default	Purpose
`checkpoint_timeout_secs`	`Option<u32>`	`None`	Per-run checkpoint-prompt timeout override (seconds); `None` resolves to the `loop.checkpoint_timeout_secs` config default (120). Set by the one-shot `--timeout` flag.

Enforcement predicates on the spec:

- `allows_tool(tool)` – true when no tool set is configured, else membership.
- `path_in_scope(path)` – true when no scope is configured, else glob match.
- `path_is_read_only(path)` – glob match against `read_only`.
- `deny_reason(tool, input) -> Option<String>` – the full restriction check (Section 9).

Invalid glob patterns are silently skipped at compile time (they can match nothing, which fails closed).

---

7. Stop conditions and termination statuses

A loop run ends when exactly one stop condition fires; the first one wins and the tracker becomes stopped, after which no further stage may run.

Condition	Trigger	Published status
Goal achieved	Model responds without tool calls (plain goal loop)	`completed`
Verification passed	Verify command exits 0	`completed` (with the verification summary attached)
Step budget reached	`max_steps` iterations consumed	`budget_exhausted`
Token budget reached	accumulated input + output tokens >= `cost_limit`	`budget_exhausted`
Verification failing with budget breached	verify keeps failing once the budget is spent	`budget_exhausted`
Unrecoverable error	Provider/client failure during turn setup or the LLM stream	`error`
Consecutive recoverable failures exceeded	more than 3 (retry allowance) consecutive recoverable failures	`error`
User interrupt	`Esc` during a loop run	`interrupted`

Note the naming: the success variant is called `GoalAchieved` internally, but the published status string is `completed` – there is no `goal_achieved` status.

The TUI renders a status-aware banner from `loop_termination_banner(status, iterations)`:

Status	Banner
`completed`	`Goal loop completed - {N} iteration(s)`
`error`	`Goal loop failed - {N} iteration(s)`
`budget_exhausted`	`Goal loop budget exhausted - {N} iteration(s)`
`interrupted`	`Goal loop interrupted - {N} iteration(s)`
other	`• Goal loop {other} - {N} iteration(s)`

When present, the message window also appends `Verification: {outcome}` and `Reason: {why}`; the log panel records one line `loop terminated · {status} · {N} iteration(s)`.

---

8. The verification gate

The verification gate runs only when the active `LoopSpec` carries a `verify_cmd`. When the model produces a response without tool calls, the loop runs the command instead of accepting the answer:

- **Pass (exit 0)** – the loop ends with status `completed` and the captured verification output summary is attached to `Event::LoopTerminated`.
- **Fail, steps remaining** – the failure output is formatted into an observation (`verification_failure_observation`) and appended as the next observation in the chat history; the loop continues so the model can act on the failure output. A success is also recorded at this point, which *resets the consecutive-failure counter* so unrelated recoverable failures do not burn the retry allowance.
- **Fail, budget breached** – the run ends as `budget_exhausted`; there is no opportunity to retry the verification.

Execution details (mirroring the Bash tool's shell handling):

Property	Value
Execution	Shell command run inside the loop's working directory
Timeout	600 seconds (`VERIFICATION_TIMEOUT`); a timeout counts as a failure
Output capture	head 8000 chars + tail 2000 chars, ASCII-truncated
Spawn error	Counts as failure (e.g. the program does not exist)
Status description	`exit code {N}` style reasons, mirroring the Bash tool

---

9. Restrictions: tool set, read-only, scope

Before every tool execution during a loop run, `deny_reason` evaluates the restriction layers in a fixed order. A `None` result means the call proceeds to the normal permission layer (allow/deny/ask rules, YOLO, etc.).

1. **Tool-set restriction (FR-008/009)** – cheapest, most structural check. Mandatory safety tools are always exempted. Message: `scope violation: tool out of scope - tool '<t>' is outside the loop's tool set (allowed: …)`.
2. **Read-only constraint (FR-021)** – for `Bash`, per-sub-command write heuristics plus redirection targets are inspected (e.g. `sed -i`, any `>` redirection); for other tools, `LOOP_WRITE_TOOLS` membership crossed with path globs. Message: `constraint violation: path '<p>' is read-only for this loop (read-only constraints: …)`.
3. **Scope boundaries (FR-022)** – every extracted path must be in scope. Message: `scope violation: path '<p>' is outside the loop's scope boundaries (...)`.

A denial becomes a tool-error observation; the loop continues, and the model sees why the call was rejected.

Supporting constants:

- `LOOP_WRITE_TOOLS` – mutating tools, including legacy aliases: `Write`, `create`, `Edit`, `multi_edit`, `multiedit`, `patch`, `apply_patch`, `append_to_file`, `update_file`, `write_file`, `rm`, `move_file`, `copy_file`, `make_directory`, `memory_store`, `memory_forget`, `memory_replace`, `memory_write`, `pdf_write`.
- `LOOP_ALWAYS_ALLOWED_TOOLS` – mandatory safety tools that stay available even under a restricted tool set: `think`, `AskUserQuestion`, `agent_complete`, `model_info`, `memory_store`, `memory_recall`, `memory_forget`, `conversation_search`, `session_search`.
- `BASH_WRITE_COMMANDS` – mutating shell commands (`rm`, `mv`, `cp`, `mkdir`, `touch`, `truncate`, `shred`, `dd`, `tee`, `chmod`, `chown`, `chattr`, `install`, …); `sed` is handled specially for its in-place `-i` flag; `git` and package managers are deliberately excluded and left to the checkpoint layer.

---

10. Checkpoints and rollback

- **Forced checkpoints.** With `checkpoints: true` (the default), a *destructive* tool call inside a loop forces an interactive checkpoint prompt even when an allow rule (or YOLO mode) would have auto-approved it. The checkpoint waits `checkpoint_timeout_secs` (default 120); a timeout is treated as denial – the intervention defaults to the safe choice.
- **Pre-loop snapshot.** `start_loop` records the git state for the workspace immediately. The file snapshot itself is armed lazily at the first-write safe point (`ensure_pre_loop_capture`), so read-only loops never pay for one. If the workspace is not inside a git repository, a notice is published: `loop workspace is not inside a git repository (FR-018); rollback will be snapshot-only`.
- **Change summary and rollback offer.** When a loop that made workspace changes terminates, the processor publishes a change summary and the TUI offers:

  ```
  Loop changes (completed, 3 iterations): 2 modified, 0 created, 0 deleted - +12 -3 lines acr
  Roll back to the pre-loop snapshot? Enter: roll back, Esc: keep changes
  ```

  The status bar shows `loop ended - Enter: roll back to pre-loop snapshot, Esc: keep changes`. A no-write loop never publishes a change summary and never offers a rollback.

Rollback decision keys

While the offer is pending, the key interceptor consumes **every** key:

- `Enter` – confirm: restore the pre-loop snapshot. Log: `rollback accepted · restoring {N} file(s)`.
- `Esc` – decline: keep all loop changes; pending captures are cleared. Message: `Rollback declined - all changes made by the loop are kept.`

If the restore reports nothing to restore, the status shows `no pending capture (nothing to restore)`; on error the capture is kept for retry.

---

11. Configuration options

The `loop` section of `ragent.json` configures the goal-loop feature. It is **exclusively** the `/loop` feature's section – it does not tune the core per-turn loop (see `docs/howtos/reactagent.md` for `agent_perf.*` / `stream.*`).

```
{
  "loop": {
    "max_steps": 512,
    "cost_limit": 200000,
    "error_retry_allowance": 3,
    "checkpoints": true,
    "checkpoint_timeout_secs": 120
  }
}
```

Key	Type	Default	Effect
`loop.max_steps`	`u32`	`512`	Maximum loop iterations; exceeding it stops the loop with status `budget_exhausted`
`loop.cost_limit`	`Option<u64>`	`None`	Maximum accumulated tokens (input + output) before the loop stops with `budget_exhausted`; `None` = no gate
`loop.error_retry_allowance`	`u32`	`3`	Consecutive recoverable-failure tolerance; the loop stops with status `error` once consecutive failures exceed this count
`loop.checkpoints`	`bool`	`true`	Force destructive-action checkpoints even when allow rules would auto-approve

Key	Type	Default	Effect
`loop.checkpoint_timeout_secs`	`u32`	`120`	Seconds a forced checkpoint prompt waits before timeout counts as denial

Precedence rules

1. An explicit `max_steps` on the loop spec (dialog) wins over everything.
2. Otherwise the per-agent `agent.<name>.max_steps` override applies, if set.
3. Otherwise the `loop.max_steps` config default (512) applies.

The same precedence applies to the cost limit (spec value > config value, with `None` meaning "no gate" when nothing is configured).

Merge semantics

Config layers are deep-merged; overlay fields that differ from the compiled defaults win. `checkpoints` is monotonic during merge: once it is `false`, it stays `false`.

The serialized config omits the `loop` section entirely when every field is at its default value.

---

12. Commands and keyboard surface

Slash commands

Command	Effect
`/loop`	Open the interactive loop setup dialog (restores a cancelled draft if any)
`/loop <agent> <goal text...>`	Start the loop immediately with the documented one-shot defaults
`/cancel`	Cancel in-flight processing
`/undo`	Snapshot rollback outside the loop flow
`/cost`	Run-cost summary for the session
`/autopilot on [--max-tokens N] [--max-time N]`	Autonomous continuation with its own budgets (separate feature)

Keyboard

Key	Effect
`Esc` (during a loop run)	Raise the loop interrupt – the loop stops after the current stage, status `interrupted`
`Enter` (while a rollback offer is pending)	Restore the pre-loop snapshot
`Esc` (while a rollback offer is pending)	Keep the loop's changes

Programmatic surface (library)

Function	Behaviour
`SessionProcessor::start_loop(&self, session_id, spec)`	Register the spec + tracker, record git state, arm interrupt flag
`request_loop_interrupt(session_id)`	Set the interrupt flag; returns whether a loop was active
`rollback_loop(session_id)`	Restore the pre-loop capture; `Ok(false)` when nothing is pending
`clear_loop(session_id)`	Remove tracker, spec, interrupt flag, and capture
`clear_loop_captures()`	Clear all pending pre-loop captures (rollback decline)

There is **no HTTP endpoint** for starting a loop; `/loop` is driven from the TUI. (`POST /sessions/{id}/messages` drives plain turns, and autopilot is the HTTP/TUI autonomous-continuation feature.)

---

13. Worked examples

13.1 Drive the test suite to green

Interactive dialog:

1. `/loop`
2. `agent`: `coder`
3. `goal`: `cargo test -p ragent-tui passes with zero failures`
4. `verify cmd`: `cargo test -p ragent-tui`
5. `max steps`: `40`
6. `Enter`

The loop edits code, runs the tool, receives failures as observations, and iterates until the verify command exits 0 or the budget is spent.

One-shot equivalent (no verify command, so the first tool-free answer ends the loop):

```
/loop coder make cargo test -p ragent-tui pass with zero failures
```

13.2 Scoped refactor with protected paths

Dialog values:

- `agent`: `coder`
- `goal`: `rename crate ragent-server to ragent-http across the workspace`
- `scope`: `crates/**, Cargo.toml`
- `read-only`: `specs/**, docs/**`
- `max steps`: `60`
- `checkpoints`: on

Writes inside `specs/` or `docs/` are denied with a `constraint violation` observation; the loop can read them but not modify them.

13.3 Read-only research sweep

Dialog values:

- `agent`: `general`
- `goal`: `produce a migration risk list for every TODO in src/`
- `read-only`: `**` (nothing writable)
- `max steps`: 30

With nothing writable the loop cannot mutate the workspace; the final tool-free answer is the deliverable and ends the loop with `completed`.

13.4 Budgeted doc-coverage pass (curl-driven session, then TUI loop)

Start a session and drive a plain turn over HTTP (the loop itself is started in the TUI):

```
curl -s -X POST http://127.0.0.1:3000/sessions \
  -H "Authorization: Bearer $RAGENT_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"directory": "."}'

curl -s -X POST http://127.0.0.1:3000/sessions/<id>/messages \
  -H "Authorization: Bearer $RAGENT_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"content": "list every public function missing doc comments in crates/ragent-agent/src/ses
```

Then, in the TUI against the same session, program the fix loop:

```
/loop coder add doc comments to every function in the missing-docs list
```

13.5 CLI one-shot, then inspect the outcome

```
ragent run --agent coder --maxsteps 50 "audit specs/agentloop against its FRs and report gaps"
```

- `--maxsteps` (single word) caps the agentic loop steps.
- `--yes` (alias `--no-prompt`) auto-approves permission prompts – note that forced loop checkpoints still interrupt destructive actions when checkpoints are on.
- `ragent serve --addr 127.0.0.1:9100` starts the HTTP API (there is no `--port` flag).

---

14. Events and telemetry

Two workspace events carry the loop lifecycle to every subscriber (TUI, SSE):

`Event::LoopTerminated` – published exactly once when a stop condition fires. Fields:

Field	Meaning
`session_id`	Session that ran the loop
`status`	Termination label: `completed`, `error`, `budget_exhausted`, `interrupted`
`iterations`	Iterations consumed

Field	Meaning
`verification`	Verification-command output summary when one ran
`reason`	Human-readable explanation for non-success terminations

`Event::LoopChangeSummary` – published when a loop that changed the workspace terminates. Fields:

Field	Meaning
`status`	Mirrors `LoopTerminated.status`
`iterations`	Iterations consumed
`files_modified`, `files_created`, `files_deleted`	Change counts
`diffstat`	e.g. `"+12 -3 lines across 4 files"`
`files`	Sorted paths, relative to the workspace root

Telemetry is recorded exactly once per loop run (`loop_telemetry_recorded` flag): the loop duration and iteration count plus the per-run tool-call tally are written through the telemetry recorder when the run terminates with a known start time. Starting a new loop resets the flag.

---

15. Related documentation

- `docs/howtos/reactagent.md` – the core per-turn ReACT agent loop (what runs underneath every `/loop` iteration), including `agent_perf.*`, `stream.*`, and compaction tuning.
- `TUI-QUICKSTART.md` – section "Goal-driven loops with `/loop`" for a quick-reference version of this guide.
- `specs/agentloop/SPEC.md` – the requirements specification (FR-001..FR-025) this feature implements.